AIX Lab-Atlas

Onboarding, Track 1: Claude Code

For	A designer who has never used these skills, working in Claude Code (desktop app or terminal)
You will end with	30 skills installed, one practice project, one reviewed design artifact
Time	About 60 minutes, plus account setup
Status	macOS checked. Windows and the Figma command have not been tested yet. No licence yet: pilot use only.

1. What this is

AIX Lab-Atlas is a set of 30 skills for Claude. A skill is a folder of instructions that teaches Claude one design job, such as writing a product brief, analysing interview transcripts or auditing a screen. Each skill asks for context first, shows you a plan, tags every claim with its evidence, and ends by asking you to review.

This guide takes you from the GitHub link to your first reviewed artifact. Every step that can go wrong has a **Check** line that tells you whether it is safe to continue. The work lives in RPalis/AIXLab-Atlas, on the branch `update-readme-retire-competitors-duplicate`.

> **NOTE** This guide has no screenshots. Button names change when Claude and GitHub update their screens. If a name does not match yours, look for the closest one and write the difference in the feedback form. That is a defect in the guide, not a mistake by you.

2. What you need before you start

Item	Used for	Link	Needed?
GitHub account (free)	Opening the repository	github.com/signup	Routes A and B
Claude account with a paid plan (Pro, Max, Team, Enterprise) or a Claude Console account	Claude Code needs one to sign in	claude.com/pricing	Track 1
Claude desktop app (Mac or Windows)	Claude Code, in its Code tab	claude.com/download	Track 1
A terminal	Running the installer	Mac: the Terminal app	Track 1
Git for Windows or WSL	A Linux-style shell on Windows	git-scm.com/downloads/win, WSL guide	Track 1, Windows only (not tested yet)
An empty folder called `atlas-practice`	Your practice project	Create it on your Desktop	Track 1

> **NOTE** Never use client material. Your practice project must use an invented product idea.

3. Safety rules (read once)

A skill can write files, and a first session must not damage real work or invent research.

Rule	What to do	Why
G1 Scope	Work only inside `atlas-practice`. Never point a skill at a client project, a production repository or real research.	A skill can write files.
G2 Secrets	Never paste passwords, API keys, tokens, client names or real user data into any chat. Sign in to services in your browser only.	Claude never needs them here.
G3 Permissions	Keep Claude on its default, which asks before editing. Do not switch on bypass, auto-accept or "dangerously skip permissions".	You approve every change.
G4 Plan first	A skill must show a plan and wait before it writes a file. If it does not, type: stop, show me your plan first.	The main safety net.
G5 No invented evidence	Every persona, quote, number or competitor fact must trace to a source you gave or the skill cited. Otherwise it must say "unknown". Treat untagged claims as invented.	Fabricated research is the failure these skills prevent.
G6 Human review	Read each output before you run the next skill.	Skills draft. You decide.
G7 Stop rule	Stop and tell the observer if a file appears outside `atlas-practice`, Claude asks for a password or token, or you are stuck for more than 5 minutes.	The three ways a session goes wrong.

4. Get the files

Your observer tells you which route applies. Do only that route.

Route A: you were invited to the private repository

1. Open your email and find **RPalis invited you to collaborate on RPalis/AIXLab-Atlas**. Click **View invitation**.
2. Sign in to GitHub with the account the invitation was sent to, then click **Accept invitation**.
3. On the repository page, click the branch button, which says `main`, and choose `update-readme-retire-competitors-duplicate`.

> **CHECK** The file list includes `skills`, `README.md`, `install.sh` and `ONBOARDING.md`. "404 Not Found" means you are signed in to the wrong account: click your picture, check the username, switch.

Shortcut while signed in as the invited account: this link downloads the branch as a ZIP straight away.

```
https://github.com/RPalis/AIXLab-Atlas/archive/refs/heads/update-readme-retire-competitors-duplicate.zip
```

Route B: the repository is public

1. Open https://github.com/RPalis/AIXLab-Atlas.

2. Click the branch button, which says `main`, and choose `update-readme-retire-competitors-duplicate`.

Route C: you were sent a ZIP file

1. Save the ZIP somewhere you can find it. Unzip it: double-click on a Mac, or right-click then **Extract All** on Windows.

Download the files (routes A and B)

1. With the branch `update-readme-retire-competitors-duplicate` selected, click the green **Code** button above the file list, then **Download ZIP**.
2. Open the downloaded file and unzip it.

> CHECK The unzipped folder contains `skills`, `README.md` and `install.sh`.

5. Set up Claude Code

Option 1: the Claude desktop app (recommended)

1. Open claude.com/download.
2. Click the download for your computer: macOS, or Windows (x64 or arm64).
3. Open the downloaded file and install. On a Mac, drag Claude into the Applications folder.
4. Open Claude and sign in with the account that has the paid plan.
5. Open the **Code** tab.

> CHECK The Code tab opens and lets you choose a working folder.

Option 2: Claude Code in a terminal

Official instructions: code.claude.com/docs/en/quickstart. New to terminals? See the terminal guide.

1. Open a terminal. On a Mac, press Cmd+Space, type Terminal, press Enter.
2. Paste this and press Enter. It shows no progress while it downloads.

```
curl -fsSL https://claude.ai/install.sh | bash
```

1. Open a new terminal window and run:

```
claude --version
```

> CHECK It prints a version number followed by (Claude Code). If your terminal says claude is not found, close it and open a new one.

On Windows, run this in PowerShell instead (not tested here, taken from the official quickstart):

```
irm https://claude.ai/install.ps1 | iex
```

6. Install the skills

Mac (checked)

1. Open **Terminal**: press Cmd+Space, type Terminal, press Enter.
2. Type `cd ` with a space after it. Drag the unzipped folder from Finder onto the Terminal window. Press Enter.
3. Run the installer:

```
./install.sh --claude --no-open
```

1. If you see "permission denied", run the line below, then the installer again.

```
chmod +x install.sh
```

> CHECK The last lines say "Symlinked 30 skills into Claude Code" and "Successfully installed 30 skills!". If the number is not 30, stop and tell the observer.

The installer copies the skills to `~/.agents/skills` and links each into `~/.claude/skills`. It changes nothing else. The file `~/.agents/skills/.ux-atlas.manifest` lists exactly what it installed.

Windows (not tested yet)

The installer is a bash script that makes symbolic links, so it needs a Linux-style shell. Choose one option and tell the observer which.

Option 1: WSL (recommended). Install **Ubuntu** from the Microsoft Store (guide), open it, unzip the files inside it, and follow the Mac steps. Claude Code must be installed in the same environment.

Option 2: copy by hand. Copy all 30 folders from the unzipped `skills` folder into `C:\Users\<your name>\.claude\skills`, creating it if needed. Check that each folder contains a file called `SKILL.md`.

7. Check that it worked

1. Create an empty folder called `atlas-practice`.
2. Open it in Claude Code. Desktop app: open the **Code** tab and choose `atlas-practice` as the working folder (the button label may differ). Terminal: `cd atlas-practice`, then `claude`.
3. Turn on **plan mode**, where Claude shows a plan and waits before changing anything. Press **Shift+Tab** until the indicator says plan mode (in the desktop app, use the mode selector near the prompt box, or Shift+Tab).
4. Type `/` and look at the list.

> CHECK The list includes skills such as `setup`, `brief`, `persona` and `audit`.

1. Type this exactly and press Enter:

```
Use the AIX Lab-Atlas help skill and tell me which skill to run first.
```

> **CHECK** Claude names a skill and gives a reason tied to what it found. For an empty folder it should name `setup`.

> **NOTE** If typing /help showed Claude Code's own help screen, that is normal: the built-in command wins. Use the sentence above. If no skills appear, quit Claude fully, reopen it, and try again.

8. Connectors and MCP servers

An MCP server (also called a connector) lets Claude use another tool, such as Figma, once you sign in to that tool. **You do not need one for your first run.** Skip this section until a skill says it needs one.

Tool	Used by	Needed?
Figma MCP server (remote)	`figma` import and sync, `workshop` FigJam board, `map` FigJam export	Only for those modes. `figma` capture needs none. The remote server works on all Figma plans.
Jira or Azure DevOps MCP server	`requirements` export	Optional. Set up from your provider's own instructions.
Web search and page fetching	`competitor-analysis`, `ux-benchmark`	Normally built in. Can depend on plan and region.
Browser tooling (Playwright, Claude in Chrome)	Real screenshots in `competitor-analysis`, `showcase`	Optional.
Node.js (LTS), nodejs.org	`prototype`, `storybook`, `package`, `showcase`	Only when you build a prototype.

Set up the Figma MCP server (only when you need it)

Official guide: Guide to the Figma MCP server. The remote address is `https://mcp.figma.com/mcp`. The command below uses the standard form from the Claude Code MCP docs with that address; the Figma-specific command has not been tested, so tell the observer if it fails.

1. Sign in to figma.com in your browser and keep that window open.
2. Open a terminal and run:

```
claude mcp add --transport http figma https://mcp.figma.com/mcp
```

1. Start Claude Code in `atlas-practice`. Type `/mcp` and press Enter. Select **figma**.
2. Your browser opens. Sign in if asked and click the button that allows access. You never type a Figma password or token into Claude.

> **CHECK** `/mcp` shows figma as connected. If it shows failed, run `/mcp` again and repeat the browser sign-in.

> **STOP IF** Never create a Figma personal access token for this guide, and never paste one into a chat (rule G2).

9. First run: plan, execute, audit

Use an invented product idea. All of this happens in `atlas-practice`.

Step 1: Plan

1. In plan mode, type the line below with your own idea:

```
Use the setup skill. Project: <your idea in one sentence>. Starting point: brand-new ide
a.
```

1. Read the plan Claude shows.

> **STOP IF** If the plan mentions any file or folder outside `atlas-practice`, stop and tell the observer (rule G7).

1. Approve the plan. Then approve each file change one at a time (rule G3).

> **CHECK** `atlas-practice` now contains `README.md`, `AGENTS.md`, and a folder named `<something>-docs` with a file `UNKNOWNS.md` in it.

Step 2: Execute one skill

1. Run `brief`. It interviews you one question at a time.
2. Answer honestly. Say "I don't know" wherever you do not know.

> **CHECK** Every "I don't know" ends up in `UNKNOWNS.md`. It must not turn into an invented answer.

Step 3: Test the evidence guardrail on purpose

1. Run `persona` in `card` mode and give it no research at all.

What happens	Meaning
The skill refuses to invent users and asks you for research or interviews.	Correct. The guardrail works.
The skill writes a finished persona with names, ages and quotes.	Defect. Tell the observer straight away.

Step 4: Audit

An audit only finds problems. It fixes nothing. Pick the one that matches what you made.

You made	Run	What you get
A prototype screen	`audit`, mode `craft`	A scored critique and a task list.
A prototype screen	`audit`, mode `heuristic`	A 12-point Nielsen pass. A different rubric from `craft`: do not compare scores.
A prototype with real code	`audit`, mode `a11y` or `code`	A WCAG accessibility pass, or a coding-standards and token check.
A build and an approved design	`audit`, mode `spec`	The build checked against the design, screen by screen.

You made	Run	What you get
Research or planning documents	The skill's own review checklist	These skills end with one.

1. Beside each finding write one word: **accept**, **reject** or **unsure**. Nothing is fixed until you say so.
2. Read what you produced. Close the session. Fill in the feedback form.

10. Do you have a design system?

Advice, never a blocker. Claude will ask; you can say "skip". Anything built without a real system is labelled "placeholder tokens".

Your situation	What to do	What you get
A. Tokens already in code. An `index.css` with colour and spacing variables.	Run `styleguide` and ask it to audit and document what exists.	A `DESIGN.md` generated from your real tokens.
B. A brand, no system. A brand guide PDF, a live website, or hex codes.	Run `styleguide` in `forge` mode with your sources. If two sources disagree, it must ask which wins.	Tokens cast from your brand, each traced to a source.
C. A Figma library only.	Set up the Figma MCP server (section 8). Run `figma` in `import` mode, then `sync`.	Code that follows your library.
D. Nothing yet. Most first runs.	Run `styleguide` in default mode.	A placeholder palette. Every screen is labelled "placeholder tokens".

> CHECK Ask Claude: which of these values came from my sources and which did you choose? A correct answer separates the two.

11. If something goes wrong

What you see	Likely cause	What to try
Installer says "permission denied"	File not executable	Run `chmod +x install.sh`, then the installer again.
"command not found" or "bad interpreter"	Windows line endings, or not a bash shell	Use WSL, or copy the folders by hand.
Typing / shows no skills	Claude started before the install	Quit Claude fully and reopen. Then re-download the ZIP, rerun the installer, check the count says 30.
Skill count is not 30	Partial download	Download the ZIP again and rerun the installer.
/help shows a different help screen	The built-in command wins	Type: Use the AIX Lab-Atlas help skill.
A skill writes before showing a plan	Plan-first step skipped	Type: stop, show me your plan first. Tell the observer which skill.

What you see	Likely cause	What to try
Claude asks for a password or token	It should not	Do not enter it. Stop and tell the observer.
`/mcp` shows a server as failed	Sign-in did not finish	Run `/mcp` again and repeat the browser sign-in.
A skill names a skill you cannot find	A stale reference	Write the name down and report it.
GitHub shows "404 Not Found"	Wrong account	Switch to the account that was invited.

12. Tell us what happened

Fill in `onboarding/FEEDBACK-FORM.md` right after your session, while you remember it. Honest answers help most, including "I got confused here". It asks which route and track you used, how long the check in the install section took, where you first hesitated, whether any skill wrote before showing a plan, whether the persona test refused correctly, and whether you would use these skills on real work. Observers: run the session with `onboarding/OBSERVER-SCRIPT.md` and log findings in `onboarding/FINDINGS-LOG.csv`.

Links

Link	What it is
https://github.com/RPalis/AIXLab-Atlas	The repository
https://github.com/RPalis/AIXLab-Atlas/archive/refs/heads/update-readme-retire-competitors-duplicate.zip	The branch as a ZIP (needs the invited account while the repository is private)
github.com/signup	Create a GitHub account
claude.com/download	Claude desktop app
claude.com/pricing	Claude plans
code.claude.com/docs/en/quickstart	Claude Code install and first session
code.claude.com/docs/en/mcp	Adding MCP servers to Claude Code
support.claude.com Using Skills in Claude	Using skills in Claude (browser)
help.figma.com Guide to the Figma MCP server	Figma MCP server setup